Bliard Samuel
Boschet Maxime

Système expert dans le domaine des Plante grasses

Sommaire:

Partie 1 : Résumé travail préliminaire **2**
1.) Domaine d'expertise choisie: 2
2.) Objectif du système expert: 2
3.) Logique générale des règles (base de faits, moteur d'inférence): 2
Partie 2 : Implémentation du moteur d'inférence **4**
1.) Implémentation moteur d'inférence: 4
2.) Structures de données utilisées: 4
A. Représentation des faits 4
B. Représentation des règles 4
3.) Fonctions de services: 5
A. Verifier-fait : vérifier si un fait existe dans la base de faits 5
B. Obtenir-valeur : récupérer la valeur d'un attribut 5
C. Evaluer-clause : tester si une clause est vraie 5
D. Executer-action : appliquer une action d'une règle 6
E. Executer-regle : appliquer toutes les actions d'une règle 6
F. Regle-activable : vérifier si une règle doit se déclencher. 6
G. Chainage-avant : le moteur d'inférence en entier 6
4.) Chaînage arrière 7
A. *Nom_Base_Fait* : 7
B. *BC* : 7
C. verif_conseil : 7
D. chainage_arriere : 7
Partie 3 : Interaction Utilisateur LISP **8**
1.) Fonctionnement de l'interaction utilisateur 8
Partie 4 : Extensions possibles : Application Unity "Save my Plant" **8**
1.) Création de l'interaction utilisateur 8
2.) UI et partie graphique de l'application 9
3.) Capture d'écran de l'application 9
Partie 5 : Usage de l'IA générative **10**
Partie 6 : Discussion et validation **10**
1.) Limites de l'implémentation 10
2.) Robustesse du moteur 11
3.) Amélioration possibles 11
Partie 7 : Remerciement **11**
Annexe : **12**
Annexe n°1 : Base de règle 12
Annexe n°2 : Base de faits 14

Partie 1 : Résumé travail préliminaire

1.) Domaine d’expertise choisie:

Nous avons conçu, formalisé et illustré le fonctionnement d’un système expert (SE) dont le domaine d’expertise est le diagnostic et l'entretien des plantes grasses d’intérieur. En effet, les plantes grasses sont des plantes d’intérieur courantes et qui, dans les dernières dizaines d’années, se sont popularisées comme plantes d'appartement. Cependant, prudence, car même s’il est facile de se procurer une de ces plantes, il est aussi très simple de mal s’en occuper, et des débutants font donc des erreurs qui peuvent causer l'endommagement de la plante. Quand ces plantes sont endommagées, il n’est pas toujours facile de faire un diagnostic et de comprendre l’origine du problème. C’est pour cela que nous avons choisi de construire un SE qui a pour but d’aider les aspirants horticulteurs à savoir prendre une décision quand leurs plantes grasses présentent des symptômes de faiblesse.

2.) Objectif du système expert:

Notre système expert a pour objectif de déduire la cause de l’état de la plante et ensuite de proposer une solution. Ce SE couvre les problèmes d’entretiens les plus courants pour une plante grasse comme des problèmes liés à la température, l'arrosage ou même l’intensité lumineuse et le type de lumière du milieu. Quand notre SE identifie le problème, il pourra parfois le reconfirmer avec d'autres éléments de la base de fait mais surtout, après le diagnostic, il proposera des solutions pour pallier au problème.

3.) Logique générale des règles (base de faits, moteur d’inférence):

<u>Base de fait :</u>

Notre Base de fait (BDF) est composée de fait de la forme suivante :
<Fréquence_Arrosage_Mois, >3>
<Humidité_Sol, trempé>
<Fréquence_Arrosage_Mois, <=1>

Cette BDF est composée de faits observés et donnés au SE par l'utilisateur mais elle est aussi complétée par le SE au fur et à mesure des règles. Par exemple, dans la règle R2 on a :

R2 : SI <Humidité_Sol, trempé> ET <Couleur_Feuille, jaune> ET <Fréquence_Arrosage_Mois, >2>
ALORS <Risque_Asphyxie, élevée>.

<Risque_Asphyxie, élevée> est ajoutée dans la base de fait et peut être utilisé dans d’autres règles comme par exemple R3 :

R3 : SI <Risque_Asphyxie, élevée> ET <Etat_Feuille, molles>
ALORS <Cause, Sur_arrosage>.

La base de fait entière est consultable en annexe n°2.

Base des règles :

L'ensemble de nos règles (détaillées dans l'annexe n°1) ont la forme :
SI [conditions] ALORS [conclusions]
[conditions] étant une ou un ensemble de faits en intersection (ET logique)
[conclusion] étant un ou plusieurs nouveaux faits produit par la règle

Par exemple pour la règle R1 on a :
R1 : SI <Fréquence_Arrosage_Mois, >3> ET <Température_Milieu_celsius, entre_10_et_20>
ALORS <Humidité_Sol, trempé>.

Donc la condition ici est que la fréquence d'arrosage est de plus de 3 fois par mois et que la température soit entre 10 et 20 degrés. La conclusion de cette règle est que le sol est trempé, donc <Humidité_Sol, trempé> est ajouté à la Base de fait.

Moteur d'inférence :

Le moteur d'inférence fonctionnera en deux étapes qui bouclent. D'abord il va chercher toutes les règles dont il a toutes les conditions dans sa base de fait puis, pour chaque règles, il va les appliquer. Après avoir fait ça, la base de fait est donc mise à jour et il va continuer à boucler jusqu'au moment où il ne peut plus appliquer de nouvelles règles.

Partie 2 : Implémentation du moteur d'inférence

1.) Implémentation moteur d'inférence:

Pour notre système expert sur les plantes, on a choisi d'utiliser le chaînage avant parce qu'il correspond simplement à la manière dont on utilise l'application. En effet, on donne les symptômes qu'on observe (couleur des feuilles, humidité du sol, fréquence d'arrosage, etc.) et le système déduit petit à petit les risques possibles puis la cause la plus probable.
Le chaînage avant part des faits connus et active automatiquement les règles qui correspondent, ce qui permet d'obtenir un diagnostic sans avoir besoin de définir un but au départ.
C'est donc la méthode la plus naturelle et la plus efficace pour analyser l'état d'une plante à partir d'observations. Néanmoins nous pouvons prendre en compte que le chaînage arrière peut également être intéressant dans le cas où un utilisateur souhaiterait confirmer la maladie de sa plante et l'action qui l'aurait soignée. Cependant ce cas serait très peu fréquent, d'où la priorité au chaînage avant dans le cadre de notre SE.

2.) Structures de données utilisées:

A. Représentation des faits

Dans notre système expert, nous avons choisi de représenter chaque fait sous forme d'une liste Lisp toujours structurée de la même manière avec le nom du fait, l'objet concerné (ici la plante) et la valeur observée.
Par exemple, (Couleur_Feuille Plante jaune) signifie tout simplement que les feuilles de la plante sont jaunes.

Tous les faits sont regroupés dans *BF*, qui est une liste contenant tous les faits de notre SE. Nous avons choisi ce format relativement simple pour, par la suite, pouvoir interagir facilement avec la base de faits dans nos fonctions.

B. Représentation des règles

Pour les règles de notre système expert, nous avons choisi une structure simple et lisible. Chaque règle est représentée comme une liste contenant :

1. Le nom de la règle (par exemple R1)
2. Une partie condition
3. Une partie action

La partie condition regroupe tous les tests qui doivent être vrais pour que la règle s'applique. Nous avons souvent utilisé AND afin de combiner plusieurs conditions. Cela nous permet de vérifier facilement plusieurs faits à la fois.

La partie action indique ce que la règle doit produire. Elle peut modifier un fait existant ou en ajouter un nouveau.

Par exemple :
(Modifier Risque_Asphyxie Plante elevee) ajoute ou met à jour ce fait quand les conditions sont réunies.

Le fait que toute les règles soit regroupées dans *BR* permet à notre moteur de les parcourir une par une pour déterminer lesquelles peuvent s'appliquer.

3.) Fonctions de services:

A. Verifier-fait : vérifier si un fait existe dans la base de faits

Cette fonction prend un triplet (ex : (Etat_Feuille Plante molles)) et regarde s'il apparaît dans la base de faits.
Elle nous sert pour les clauses simples. Ce sont celles qui ne contiennent pas de comparaison numérique, c'est à dire, seulement un fait à vérifier.

B. Obtenir-valeur : récupérer la valeur d'un attribut

Cette fonction parcourt la base de faits et retourne la valeur associée à un attribut (ex : température, humidité…).
Elle nous est nécessaire pour évaluer les clauses du type :

(> Température_Milieu_celsius 25)

Elle nous donne la valeur actuelle dans la BDF (par exemple: 18 °C) que l'on pourra ensuite comparer.

C. Evaluer-clause : tester si une clause est vraie

C'est une fonction importante qui analyse une clause de la partie "SI" d'une règle et détermine si elle est satisfaite ou non.
Elle nous permet de gérer plusieurs cas:

Les comparaisons numériques (<, >, <=)

Les opérateurs logiques AND et OR (on fait un appel récursif à evaluer-clause)

Les règles de triplets classiques (que l'on vérifie avec verifier-fait)

C'est cette fonction qui nous dit si une condition est vraie ou non selon la base de faits que l'on a à un moment donné.

D. Executer-action : appliquer une action d’une règle

La fonction executer-action nous permet d’appliquer les effets d’une règle sur la BDF. Une action peut nous amener soit à ajouter un nouveau fait, soit à modifier un fait existant.

Par exemple, nous pouvons ajouter un conseil comme (Ajouter Conseil Plante Augmenter_Arrosage) ou modifier une mesure, comme (Modifier Température_Milieu_celsius Plante + 2) ou encore remplacer directement une valeur comme par exemple (Modifier Etat_Feuille Plante molles).

E. Executer-regle : appliquer toutes les actions d’une règle

Une règle peut contenir plusieurs actions dans la partie “ALORS”. De ce fait, cette fonction nous permet de les exécuter les unes après les autres en utilisant executer-action et en mettant à jour la BDF au fur et à mesure.

F. Regle-activable : vérifier si une règle doit se déclencher.

Ici, nous regardons toutes les clauses de la partie “SI” d’une règle. Cette fonction nous renvoie vrai seulement si toutes les clauses sont satisfaites.

Elle utilise evaluer-clause pour tester chaque condition séparément.

G. Chainage-avant : le moteur d’inférence en entier

C’est la fonction principale, elle applique le chaînage avant sur l'ensemble des règles.

La fonction parcourt les règles, elle déclenche les règles dont les conditions sont vraies puis elle met à jour la BDF. Elle recommence ce cycle tant que de nouvelles règles deviennent activables.

La fonction s’arrête quand plus aucune règle ne peut produire de nouvelle conclusion, et retourne alors la BDF.

4.) Chaînage arrière

Bien que le chaînage arrière soit bien moins pertinent dans notre cas pour un utilisateur, il est tout de même intéressant de s'y pencher. Dans notre SE, le fonctionnement et la logique du chaînage arrière contiennent quelques différences avec celui du chaînage avant. Ce chaînage est basé sur deux fonctions et deux variables globales (*Nom_Base_Fait* et *BC* la base de conseils) définies dans le code. Ce chaînage arrière est basé sur exactement les même règles cependant, le format des règles pour le chaînage avant étant complexe, il a été simplifié pour le chaînage arrière dans le variable *BR_CA* qui est sous le format : (Conseil ou fait_intermediaire1, fait_intermediaire2 ou fait de base, … fait_intermediareN ou fait de base N).

A. *Nom_Base_Fait* :

Cette variable sert à répertorier tous les faits possibles que l'utilisateur pourrait rentrer (ex : feuille jaune de la plante ⇒ Couleur_jaune stocké). Les noms sont répertoriés car il faut les différencier des faits intermédiaires dans le chaînage arrière. Par exemple : Asphyxie_elevee est un fait intermédiaire que l'utilisateur ne peut pas directement donner. Pour ce cas, on voudrait afficher les faits liés qui sont un arrosage excessif et une couleur des feuilles qui serait jaune.

B. *BC* :

Cette fonction répertorie tous les acronymes des conseils que la SE peut donner. Elle sert pour le Menu, afin que l'utilisateur choisisse un des conseils à partir duquel il veut faire le chaînage arrière. Elle est utilisée dans la fonction affichage_possibilite qui affiche tous les acronymes des conseils de la SE.

C. verif_conseil :

Cette fonction demande à l'utilisateur à partir de quel conseil il veut faire le chaînage arrière puis vérifie que l'utilisateur rentre un conseil qui existe. Si c'est le cas, il lance le chaînage arrière.

D. chainage_arriere :

Cette fonction boucle sur la variable locale fait_intermediaire qui contient au début du chaînage arrière seulement le conseil. À chaque iteration, on regarde quel fait intermédiaire ou fait de base a pu produire le conseil/fait intermédiaire. Si c'est un fait de base, il est mis dans la liste qui sera renvoyée fait_trouvee. Si c'est un fait intermédiaire, il sera rajouté dans la liste intermédiaire et traité à son tour. Dans ce cas, il sera aussi ajouté à la liste fait traitée pour vérifier qu'on ne l'a pas déjà traité auparavant (pour ne pas avoir de doublons ou boucler indéfiniment). Une fois la liste fait_intermediaire vide, on renvoie la liste des faits de base et on l'affiche pour l'utilisateur.

Partie 3 : Interaction Utilisateur LISP

1.) Fonctionnement de l'interaction utilisateur

L'interaction utilisateur est assez minimaliste. Quand on lance le code LISP on a un menu avec 3 choix qui apparaissent :

- Chaînage avant (diagnostic à partir des symptômes)
- Chaînage arrière (vérifier les conditions d'un conseil)
- Quitter le programme

La première option est le but premier de notre SE par chaînage avant, c'est à dire que le SE demande un par un des symptômes pour alimenter sa base de fait puis un chaînage avant est appliqué pour donner 1 ou plusieurs conseils à l'utilisateur.

Partie 4 : Extensions possibles : Application Unity "Save my Plant"

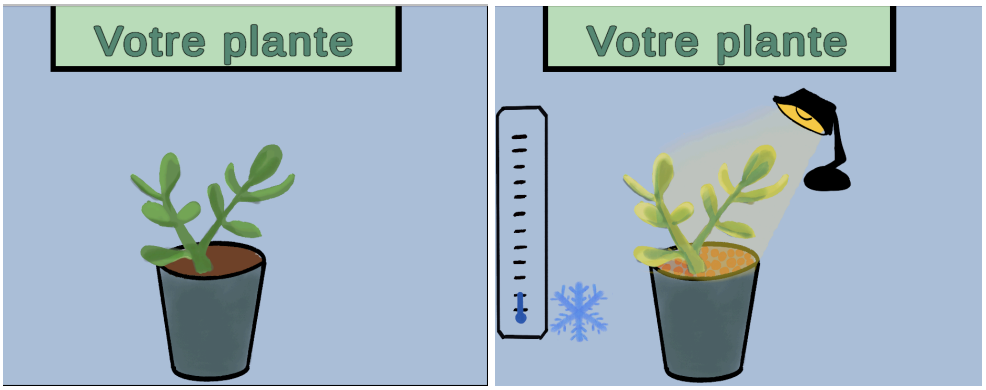
1.) Création de l'interaction utilisateur

Nous voulions absolument faire une interface graphique pour avoir une SE utilisable par tous. La première idée était d'utiliser Tkinter, l'interface graphique intégrée à python, cependant, après plusieurs tests, il était très compliqué de faire une interface jolie et compréhensible par l'utilisateur. Ayant fait plusieurs projets utilisant Unity (SI28 et personnellement aussi) et sachant que c'est un outil beaucoup plus puissant et avec lequel il serait possible de faire une jolie application contenant toutes les fonctionnalités que nous voulions, nous avons finalement décidé de l'utiliser.

Lors de la dernière phase du développement, nous avons remarqué qu'il manquait parfois des conseils, et qu'il y avait des cas où la plante n'était pas en bonne santé vu les faits donnés par l'utilisateur mais que le SE ne le détectait pas, par manque de règles. Nous avons donc rajouté des règles qui sont détaillées dans l'annexe n°1 dans la partie "règles ajoutées" (elles sont marquées en bleu). Ces règles ont aussi été ajoutées dans la version LISP de notre SE. À noter que dans l'application, il y a quelques détails et règles ajoutées qui n'ont pas été implémentés dans la version LISP, rendu en partie possible par la flexibilité de Unity et du C#.

2.) UI et partie graphique de l’application

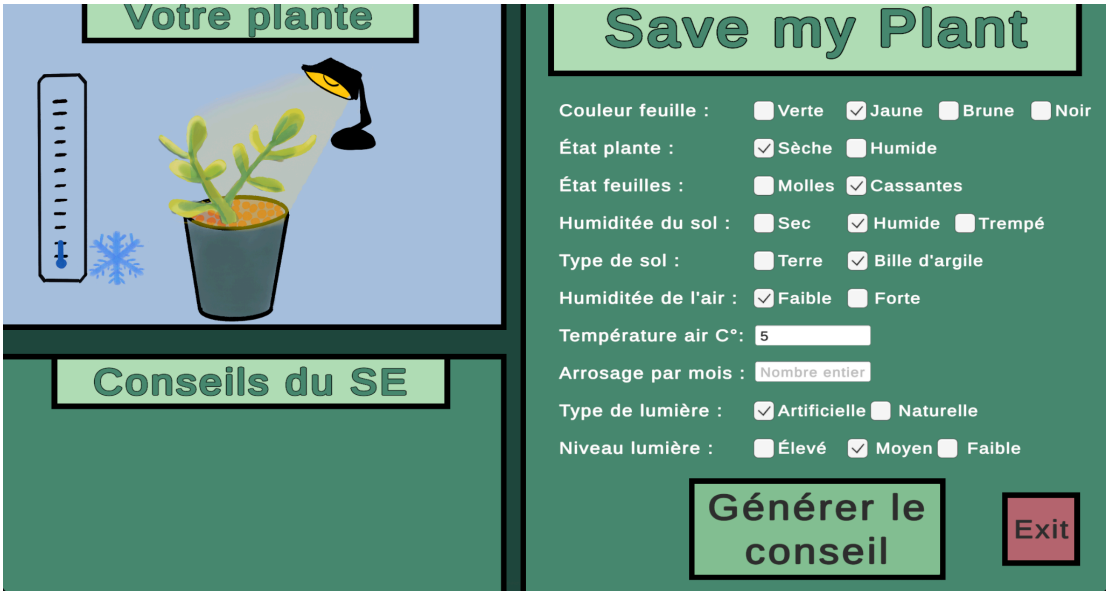
Comme nous voulions une application utilisable par tous et visuellement plaisante, nous avons eu l’idée de faire un système dans lequel une image de la plante apparaissait dans la partie gauche de l’écran et s'adaptait aux différents faits donnés par l’utilisateur. Par exemple, voici la plante de base à gauche, puis la plante adaptée à droite, après que l'utilisateur ait rentré les spécificités de sa plante.



Ces dessins ont été réalisés par ma sœur (Esther BLIARD) sur tablette graphique pour une totalité de 27 images superposées qui s'affiche et disparaissent en fonction des faits cochés par l'utilisateur. Notre talent graphique étant assez limité, nous la remercions grandement pour ces illustrations.
Nous avons décidé d’appeler cette application “Save my plant” en référence à la frustration et le désespoir que peut créer le fait de s’occuper de plantes lorsque l’on est débutant dans la matière. Cette application est disponible pour macOS et Windows dans le fichier nommé Save my plant SE.

3.) Capture d’écran de l’application

Voici une capture d’écran de l’interface de l’application :



Partie 5 : Usage de l’IA générative

Nous avons utilisé l’IA avec notamment ChatGPT pour nous aider dans l’écriture de code LISP redondant. Un des premiers exemples est l’implémentation de la base de faits et la base de règles. Lors de la première implémentation en LISP, le code plantait car il y avait des caractères accentués. Nous avons donc demandé à ChatGPT de remplacer tous les caractères accentués par des caractères non accentués.

L’IA a aussi été utilisée dans la compréhension de certaines fonctions LISP comme (format) dans l’affichage final pour l’utilisateur.

Elle a également été utilisée dans le cadre du développement en C# notamment pour gérer l’UI, Unity a récemment implémenter beaucoup de nouvelles méthodes pour gérer tout le système UI, donc l’IA a beaucoup aidé à comprendre les nouveaux fonctionnement des éléments de text (Text Mesh Pro) et de bouton (TMP button). Mais aussi à débuguer mon code lié à la caméra pour l’adaptation de la taille de l’application (quand elle est en mode windowed).

En résumé, l'utilisation de L’IA a été assez limitée. Elle s’est résumée à faire des tâches redondantes ou non principales pour accélérer le projet de façon significative sans pour autant limiter la reflexion. Notamment quand nous l’avons utilisé comme programme spécialisé par exemple dans le cas des accents où L’IA les a retirés pour nous, nous aurions pu coder un programme en C ou autre langage pour le faire à notre place sur les centaines de ligne de code mais cela aurait pris du temps pour une chose qui n’est pas le but premier du TP.
Nous estimons en temps gagné que l'IA a participé entre 4~7% mais elle n'a nullement participé dans la réflexion derrière le projet.
Cette utilisation de l’IA semble être responsable dans la mesure où elle ne remplace pas la réflexion mais automatise des tâches redondantes et accélère certains points dans l’apprentissage (par exemple dans l’explication de certaines fonctions LISP).

Partie 6 : Discussion et validation

1.) Limites de l’implémentation

Notre implémentation répond au problèmes de base d’un utilisateur pour une plante grasse mais elle a ses limites. Nous avons au total une trentaine de règles, cela ne couvre absolument pas tous les problèmes qu’une plante grasse pourrait rencontrer. Par exemple, les problèmes liés aux parasites et aux insectes n’ont pas pu être abordés avec un projet restreint comme celui-ci.

2.) Robustesse du moteur

Nous avons essayé au mieux de rendre notre moteur robuste pour ne pas frustrer l'utilisateur. Par exemple, quand l'utilisateur ne rentre pas le bon conseil à tester, dans le chaînage arrière le programme lui en informe et il a l'opportunité de réessayer et de le modifier, à la place de juste avoir un programme qui plante. Nous aurions pu pousser la chose encore plus loin dans la version LISP en corrigeant les fautes de l'utilisateur comme par exemple remplacer ses caractères accentués par des caractères non accentués.
Dans la version Unity le moteur est encore plus robuste puisqu'il ne faut plus taper le nom des symptômes mais seulement les cocher donc ici, pas d'erreur qui pourrait être commise par l'utilisateur.

3.) Amélioration possibles

Nous avons évoqué plus tôt le fait que notre SE soit limité par le nombre de règles. Il aurait été possible pour nous d'ajouter plus de règles notamment par IA, à laquelle nous aurions pu faire la requête de générer une centaine de règles pour couvrir plus de cas différents. C'est une chose que nous avons essayé de faire cependant, nous avons rencontré le problème lié aux hallucinations des IA où l'IA, par manque de données ou pour d'autres raisons, génère des choses factuellement fausses. De ce fait, pour pouvoir utiliser ce que l'IA avait généré, nous aurions dû vérifier une à une les centaines de règles générées ce qui aurait pris énormément de temps. Toutefois, cela aurait pu être une possibilité viable sur un projet plus long.
L'idéal aurait été de pouvoir parler à un professionnel du milieu pour créer ces différentes règles, ce qui est une piste d'amélioration possible pour notre SE.
Une autre amélioration possible serait l'élargissement des "connaissances" de notre SE à d'autres types de plantes. En effet, notre SE est spécialisé dans les plantes grasses, mais il serait intéressant pour un utilisateur d'avoir une même application pour toutes ses plantes, qu'elles soient grasses ou non.

Partie 7 : Remerciement

Merci à :

- Abel Marie-Helène, Domitile Lourdaux et Nader Abdelmalek pour l'apprentissage du LISP qui a servi dans ce projet.
- Serge Bouchardon, pour le guidage lors de l'apprentissage de l'utilisation de Unity.
- Esther Bliard, pour toutes les illustrations réalisées pour l'application et le projet "Save my plant".
- Christophe Bliard, pour avoir corrigé mes erreurs de recherches liées aux plantes grasses lors de l'établissement de certaines règles.

Annexe :

Annexe n°1 : Base de règle

Arrosage :

R1 : SI <Fréquence_Arrosage_Mois, >3> ET <Température_Milieu_celsius, entre_10_et_20>
ALORS <Humidité_Sol, trempé>.

R2 : SI <Humidité_Sol, trempé> ET <Couleur_Feuille, jaune> ET <Fréquence_Arrosage_Mois, >2>
ALORS <Risque_Asphyxie, élevée>.

R3 : SI <Risque_Asphyxie, élevée> ET <Etat_Feuille, molles>
ALORS <Cause, Sur_arrosage>.

R4 : SI <Fréquence_Arrosage_Mois, <=1> ET <Humidité_Sol, sec> ET <Etat_Feuille, cassantes>
ALORS <Cause, Sous_arrosage>.

R5 : SI <Fréquence_Arrosage_Mois, <=1> ET <Température_Milieu_celsius, >25>
ALORS <Risque_Sécheresse, élevée>.

Température :

R6 : SI <Humidité_Air, faible> ET <Température_Milieu_celsius, >30> ET <Couleur_Feuille, jaune>
ALORS <Cause, Stress_Chaleur>.

R7 : SI <Température_Milieu_celsius, <10> ET <Couleur_Feuille, noir>
ALORS <Cause, Stress_Froid>.

Lumière :

R8 : SI <Niveau_Lumière, faible> ET <Couleur_Feuille, jaune>
ALORS <Cause, Manque_de_Lumière>.

R9 : SI <Cause, Manque_de_Lumière> ET <Lumière, artificielle>
ALORS <Cause, Manque_de_Soleil>.

R10 : SI <Niveau_Lumière, élevée> ET <Couleur_Feuille, brune> ET <Etat_Plante, sèche>
ALORS <Cause, Trop_de_Luminosité>.

Conseils :

R11 : SI <Cause, Sur_arrosage> ALORS <Conseil, Stopper_Arrosage_Temporairement>.

R12 : SI <Risque_Asphyxie, élevée> ALORS <Conseil, Réduire_Arrosage>.

R13 : SI <Cause, Sous_arrosage> ALORS <Conseil, Augmenter_Arrosage>.

R14 : SI <Risque_Sécheresse, élevée> ALORS <Conseil, Augmenter_Arrosage>.

R15 : SI <Cause, Stress_Chaleur> ALORS <Conseil, Changer_Le_Milieu>.

R16 : SI <Cause, Manque_de_Lumière> ALORS <Conseil, Augmenter_Luminosité>.

R17 : SI <Cause, Trop_de_Luminosité> ALORS <Conseil, Changer_Milieu>
ET <Conseil, Diminuer_Luminosité>.

R18 : SI <Cause, Manque_de_Lumière> ET <Cause, Manque_de_Soleil> ALORS <Conseil, Mettre_Au_soleil>.

Voici les règles en plus, rajoutées pour enrichir notre base de donée (en bleu) :

R19 : SI <Température_Milieu_celsius, >25> ET <Humidité_Air, élevée> ALORS <Cause, Choc_Thermique>.

R20 : SI <Type_Sol, terre> ET <Humidité_Sol, humide OU trempé> ALORS <Cause, Mauvais_Drainage>.

R21 : SI <Type_Lumière, naturelle> ET <Niveau_Lumière, élevée> ET <Couleur_Feuille, noir> ALORS <Cause, Brûlure_Soleil_Direct>.

R22 : SI <Niveau_Lumière, faible> ET <Etat_Feuille, molles> ALORS <Cause, Étiolement>.

R23 : SI <Température_Milieu_celsius, <5> ALORS <Cause, Gel>.

R24 : SI <Température_Milieu_celsius, entre_10_et_15> ET <Fréquence_Arrosage_Mois, >1> ALORS <Cause, Dormance_Hivernale>.

R25 : SI <Température_Milieu_celsius, >35> ALORS <Conseil, Réduire_Température>.

R26 : SI <Humidité_Air, faible> ET <Température_Milieu_celsius, >28> ET <Etat_Plante, sèche> ALORS <Cause, Évaporation_Excessive>.

R27 : SI <Couleur_Feuille, noir> ET <Humidité_Sol, sec> ALORS <Cause, Excès_Engrais>.

R28 : SI <Etat_Plante, humide> ET <Type_Sol, bille_d'argile> ALORS <Cause, Besoin_Rempotage>.

R29 : SI <Cause, Gel> ALORS <Conseil, Rentrer_Plante_Urgence>.

R30 : SI <Cause, Brûlure_Soleil_Direct> OU <Cause, Trop_de_Luminosité> ALORS <Conseil, Éloigner_Zone_Ensoleillée>.

R31 : SI <Cause, Manque_de_Soleil> ALORS <Conseil, Exposition_Soleil_Naturel>.

R32 : SI <Cause, Étiolement> ALORS <Conseil, Augmenter_Luminosité>.

R33 : SI <Cause, Choc_Thermique> OU <Cause, Stress_Chaleur> ALORS <Conseil, Aérer_Pièce>.

Annexe n°2 : Base de faits

Base de faits possible c'est à dire que ceci n'est pas un exemple de ce que l'utilisateur pourrait rentrer mais, toutes les possibilités d'informations que l'utilisateur peut fournir à la base de fait. Donc, les différentes informations qui peuvent être saisies par l'utilisateur pour la base de fait sont :

liée directement à la plante :
Couleur_Feuille : verte, jaune, brune, noir
Etat_Plante : sèche, humide
Etat_feuilles : molles, cassantes

liée au sol :
Humidité_Sol : trempé, humide, sec
Type_Sol : terre, bille d'argile

liée au milieu :
Humidité_Air : faible, forte
Température_Milieu_celsius : entier positif
Fréquence_Arrosage_Mois : flottant positif
type_lumière : artificielle, naturelle
Niveau_Lumière : élevée, moyen, faible